

Experimental Study of the Social Force Model in Evacuation Dynamics

Hugo Pérez de Albéniz

Marc Nebelung Murciano

May, 2025

Abstract

We present an experimental investigation of the Social Force Model (SFM) for pedestrian dynamics in emergency evacuation scenarios, implemented in the open-source Cromosim simulator. By systematically sweeping five fundamental SFM parameters—crowd size, door width, desired walking speed, repulsion strength, and relaxation time—we quantify their individual and combined effects on four key evacuation metrics: total egress time, flow rate through the exit, collision rate, and local exit-area density. Our results reveal clear regime transitions: evacuation time and throughput scale nearly linearly with population until door capacity is reached; desired speed and repulsion exhibit optimal mid-range values that minimize both delay and contact events; and reaction time governs the trade-off between orderly flow and jamming. We compare these findings to empirical studies and alternative modeling work, demonstrating broad agreement with the fundamental diagram of pedestrian flow, while highlighting discrepancies arising from unmodeled decision-making phenomena. Finally, we critically assess the strengths and limitations of the SFM and Cromosim for realistic evacuation analysis, and propose directions for integrating cognitive behaviors and validation against field data.

Contents

1	Introduction	3
2	The Social Force Model	3
2.1	Driving Force Toward the Goal	4
2.2	Repulsive Effects from Other Pedestrians	4
2.3	Repulsive Effects from Walls and Obstacles	4
2.4	Optional Attractive Effects	4
2.5	Perception-Weighted Forces	5
2.6	Total Social Force	5
2.7	Motion Equation	6
3	Implementation of the Social Force Model in Cromosim	6
3.1	Driving Force Implementation	6
3.2	Repulsive Forces	7
3.3	Managing Movement and Collisions	10
3.4	Updating Pedestrian Positions	11
4	Methodology	12
4.1	Domain Design	12
4.2	Simulation Design	13
4.3	Tested SFM Parameters	14
4.4	Evacuation Metrics	15
5	Results	17
5.1	Variations of Number of People	17
5.2	Variations of Door Width	18
5.3	Variations of Desired Speed	19
5.4	Variations of Repulsion Strength	20
5.5	Variations of Relaxation Time	21
6	Discussion and Limitations	21
7	Conclusions	23
8	References	23

1 Introduction

Efficient and safe evacuation of crowded spaces—such as theaters, airports, and stadiums—remains a paramount concern in civil engineering and public safety. While architectural design and signage provide the physical framework for egress, human behavior under stress ultimately governs whether an evacuation proceeds smoothly or devolves into dangerous congestion and panic. The Social Force Model (SFM), first introduced by Helbing and Molnár (Helbing and Molnár 1995), offers a flexible, physics-inspired approach to capture how individual motivations and local interactions give rise to collective crowd dynamics, including self-organized lane formation, arching at narrow exits, and the counterintuitive “faster-is-slower” effect (Helbing and Molnár 1995).

In this work, we adopt an experimental perspective to probe the SFM’s parameter space using Cromosim (Cromosim 2018), a Python-based simulator that implements Helbing’s original force laws in a two-dimensional domain. By varying the number of agents, door width, desired speed, inter-agent repulsion strength, and relaxation time, we measure their impact on four evacuation metrics: total evacuation time, instantaneous flow rate, collision frequency, and exit-zone density. Our goal is twofold. First, we aim to map out the critical thresholds and regime changes that govern evacuation performance in the SFM framework. Second, we seek to relate these simulation-based insights to empirical observations from crowd experiments and to evaluate the methodological adequacy of the basic SFM—particularly as implemented in Cromosim—for realistic evacuation planning.

The remainder of this paper is organized as follows. Section 2 reviews the theoretical foundations of the Social Force Model and its implementation in Cromosim (Cromosim 2018). Section 3 describes our simulation setup and the evacuation metrics under study. Section 4 presents the results of our parameter sweeps, highlighting key inflection points and optimal parameter regions. Section 5 discusses the alignment of our findings with prior empirical and modeling work, and critically examines the strengths and limitations of the SFM and Cromosim. We conclude in Section 6 with recommendations for future model extensions and validation efforts.

2 The Social Force Model

The Social Force Model describes pedestrian motion using the concept of internal motivations represented as forces. These “social forces” are not physical in the classical sense, but rather reflect a pedestrian’s behavioral tendency to move toward a goal, avoid collisions, and maintain personal space.

More specifically the terms it uses can be categorized into three groups:

- a term describing the acceleration towards the desired velocity of motion.
- terms reflecting that a pedestrian keeps a certain distance to other pedestrians and borders.
- a term modeling attractive effects

All explanations in the following sections are paraphrased and derived from (Helbing and Molnár 1995)

2.1 Driving Force Toward the Goal

Each pedestrian has a destination and attempts to move toward it at a desired speed v_α^0 . If undisturbed, the pedestrian moves in the direction $\vec{e}_\alpha(t)$ (unit vector toward the next waypoint). If disturbed, the relaxation - which is the time it takes to reach the original desired velocity (desired speed times direction $\vec{e}_\alpha(t)$) again - is modeled by the acceleration term of the form:

$$\vec{F}_\alpha^0 = \frac{1}{\tau_\alpha}(v_\alpha^0 \vec{e}_\alpha - \vec{v}_\alpha)$$

Here, τ_α is the relaxation time, and $\vec{v}_\alpha$ is the current velocity of pedestrian α .

2.2 Repulsive Effects from Other Pedestrians

To avoid collisions, pedestrians experience repulsive forces from others. This repulsion depends on the perceived distance and direction. The force from another pedestrian β is given by:

$$\vec{f}_{\alpha\beta} = -\nabla_{\vec{r}_{\alpha\beta}} V_{\alpha\beta}(b)$$

where $V_{\alpha\beta}(b)$ is a potential function decreasing with distance b , and $\vec{r}_{\alpha\beta} = \vec{r}_\alpha - \vec{r}_\beta$ is the relative position. The term b (semi-minor axis of the elliptical interaction area) is defined as:

$$2b = \sqrt{(\|\vec{r}_{\alpha\beta}\| + \|\vec{r}_{\alpha\beta} - v_\beta \Delta t \vec{e}_\beta\|)^2 - (v_\beta \Delta t)^2}$$

This elliptical perception accounts for the fact that pedestrians are more sensitive to what is in front of them than behind.

2.3 Repulsive Effects from Walls and Obstacles

Similarly, pedestrians avoid walls and other objects, which produce a repulsive force:

$$\vec{F}_{\alpha B} = -\nabla_{\vec{r}_{\alpha B}} U_{\alpha B}(\|\vec{r}_{\alpha B}\|)$$

Here we take the negative gradient of the potential energy $U_{\alpha B}$, which points in the direction of steepest decrease (away from the wall or objective). $U_{\alpha B}$ is a monotonic decreasing potential meaning that it gets smaller as the agent moves away from the obstacle or wall and it depends on $\vec{r}_{\alpha B}$ which is the vector from pedestrian α to the nearest point on the wall or obstacle B .

2.4 Optional Attractive Effects

Attractive forces (e.g., toward friends or points of interest) can also be included as:

$$\vec{f}_{\alpha i} = -\nabla_{\vec{r}_{\alpha i}} W_{\alpha i}(\|\vec{r}_{\alpha i}\|, t)$$

where $\|\vec{r}_{\alpha i}\|$ is the distance between pedestrian α and target i , $W_{\alpha i}(\|\vec{r}_{\alpha i}\|, t)$ the attractive (monotonic increasing) potential, decreasing with time t and $-\nabla_{\vec{r}_{\alpha i}}$ the negative gradient operator indicating direction of steepest decrease of W so this time towards the objective or person of interest.

These effects are often omitted in evacuation scenarios as getting out of the building is the priority and main interest.

2.5 Perception-Weighted Forces

In reality, pedestrians do not respond equally to everything around them. Things in front are perceived more strongly than things behind one. To account for this, the Social Force Model includes a perception mechanism through direction-dependent weighting of forces.

Let $\vec{f}$ be an interaction force (repulsive or attractive), and let $\vec{e}$ be the pedestrian's desired direction of motion. The perception weight is defined as:

$$w(\vec{e}, \vec{f}) = \begin{cases} 1 & \text{if } \vec{e} \cdot \vec{f} \geq \|\vec{f}\| \cos \varphi \\ c & \text{otherwise} \end{cases}$$

where φ is half the pedestrian's field of vision (typically $\varphi \approx 100^\circ$) and c is a reduction factor for rear perception (e.g., $c = 0.5$). The dot product $\vec{e} \cdot \vec{f}$ determines whether the force is within the field of vision.

Using this, each interaction force is scaled to give us the "perception-weighted force":

$$\vec{F}_{\alpha\beta} = w(\vec{e}_\alpha, -\vec{f}_{\alpha\beta}) \cdot \vec{f}_{\alpha\beta}, \quad \vec{F}_{\alpha i} = w(\vec{e}_\alpha, \vec{f}_{\alpha i}) \cdot \vec{f}_{\alpha i}$$

2.6 Total Social Force

All these contributions are summed to get the total social force:

$$\vec{F}_\alpha(t) = \vec{F}_\alpha^0 + \sum_{\beta} \vec{F}_{\alpha\beta} + \sum_B \vec{F}_{\alpha B} + \sum_i \vec{F}_{\alpha i}$$

Fluctuations may be added to represent random behavior or indecision.

As a result, the total motivation to change behaviour, i.e., the acceleration of a pedestrian α , is given by:

$$\frac{d\vec{w}_\alpha}{dt} = \vec{F}_\alpha(t) + \text{fluctuations}$$

where $\vec{w}_\alpha$ is the preferred velocity and $\vec{F}_\alpha(t)$ is the sum of various social forces.

2.7 Motion Equation

Since the actual speed is limited by a pedestrian's maximal acceptable speed $v_\alpha^{\max}$, we will assume that the realized motion is given by:

$$\frac{d\vec{r}_\alpha}{dt} = \vec{v}_\alpha(t) := \vec{w}_\alpha(t) \cdot g\left(\frac{v_\alpha^{\max}}{\|\vec{w}_\alpha\|}\right)$$

with

$$g\left(\frac{v_\alpha^{\max}}{\|\vec{w}_\alpha\|}\right) = \begin{cases} 1 & \text{if } \|\vec{w}_\alpha\| \leq v_\alpha^{\max} \\ \frac{v_\alpha^{\max}}{\|\vec{w}_\alpha\|} & \text{otherwise} \end{cases}$$

The velocity of each individual - defined as the change in position with respect to time - is defined by the desired velocity $\vec{w}_\alpha(t)$ times the function g . The function g checks if the desired velocity is greater than the maximum velocity. If so it results in $\frac{v_\alpha^{\max}}{\|\vec{w}_\alpha\|}$ which brings the whole equation down to $v_\alpha^{\max}$ and otherwise the equation results in the original value of $\vec{w}_\alpha(t)$. This process ensures that pedestrians do not exceed the maximum acceptable speed $v_\alpha^{\max}$.

3 Implementation of the Social Force Model in CROMOSIM

The CROMOSIM library implements the Social Force Model through two core Python files: `domain.py` (CROMOSIM 2025a) handles the spatial domain and navigation, while `micro.py` (CROMOSIM 2025b) manages pedestrian interactions and force calculations.

3.1 Driving Force Implementation

The theoretical driving force:

$$\vec{F}_\alpha^0 = \frac{1}{\tau_\alpha}(v_\alpha^0 \vec{e}_\alpha - \vec{v}_\alpha)$$

is implemented using a pathfinding approach. The desired velocity field is precomputed using the Fast Marching Method:

```
1 # domain.py - Destination velocity calculation
2 def people_desired_velocity(self, xyr, people_dest):
3     Vd = np.zeros((xyr.shape[0], 2))
4     for dest_name in np.unique(people_dest):
5         ind = np.where(people_dest == dest_name)[0]
6         scale = self.destinations[dest_name].velocity_scale
7         Vd[ind,0] = xyr[ind,3] * scale *
8             ↪ self.destinations[dest_name].desired_velocity_X[II[ind],JJ[ind]]
9         Vd[ind,1] = xyr[ind,3] * scale *
10             ↪ self.destinations[dest_name].desired_velocity_Y[II[ind],JJ[ind]]
```

```
return Vd
```

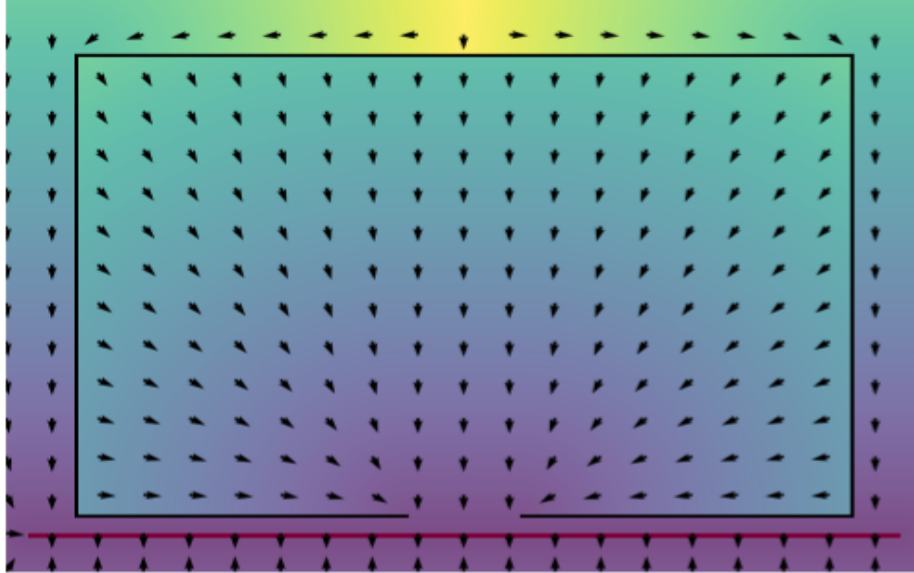
Cromosim replaces the theoretical relaxation term with destination-based navigation fields. The Fast Marching Method precomputes optimal paths by solving the Eikonal equation:

$$|\nabla D(\vec{r})| = 1/f(\vec{r})$$

where $D(\vec{r})$ is the travel time field and $f(\vec{r})$ encodes walking speeds. The desired velocity derives from:

$$\vec{V}_d = v_0 \cdot s \cdot (-\nabla D / \|\nabla D\|)$$

Figure 1: Desired Direction



This graph represents the desired direction from every position inside the room. These directions would be exactly followed by every agent in the room if no one disturbed each other and no collisions happened (which is not the case). The colors indicate how far away a point is from the red exit line at the bottom, accounting for the walls and obstacles.

3.2 Repulsive Forces

The repulsive potential:

$$\vec{f}_{\alpha\beta} = -\nabla V_{\alpha\beta}(b)$$

is computed with distance-based exponential decay and angular weighting:

```

1  # micro.py - Force computation
2  def compute_forces(F, Fwall, xyrv, contacts, U, Vd, lambda_, delta, k, eta):
3      for ic in np.arange(Nc):
4          i = int(contacts[ic,0]); j = int(contacts[ic,1])
5          dij = contacts[ic,2]; eij_x = contacts[ic,3]; eij_y = contacts[ic,4]
6
7          # Angular perception weighting
8          norm_Vdi = np.linalg.norm(Vd[i,:])
9          if norm_Vdi > 0:
10             theta_ij = np.arccos(np.dot(Vd[i,:], [eij_x,eij_y])/norm_Vdi)
11             omega_ij = lambda_ + (1-lambda_)*(1+np.cos(theta_ij))/2
12         else:
13             omega_ij = 1
14
15         # Exponential repulsion + linear overlap correction
16         fij = -omega_ij * F * np.exp(-dij/delta) + k * min(dij, 0.0)
17         Forces[i,0] += fij * eij_x
18         Forces[i,1] += fij * eij_y

```

Repulsive Forces Between Pedestrians

The model handles repulsion between pedestrians using two main components. First, there's a social force that makes people keep comfortable distances from each other. This force gets weaker as pedestrians move farther apart, following an exponential decay pattern.

Second, there's a physical force that only kicks in when pedestrians actually touch or overlap. This acts like a soft spring that pushes them apart. To find which pedestrians are close enough to interact, the code uses an efficient spatial search system called cKDTree. This helps the simulation run quickly even with large crowds.

The model also accounts for human perception - we're more aware of things in front of us than behind us. A special parameter controls how much pedestrians notice others behind their backs.

The model implements directional awareness through a weighting factor ω_{ij} in the social repulsion force:

$$\vec{f}_{\alpha\beta}^{\text{social}} = -\omega_{\alpha\beta} \cdot A e^{-b_{\alpha\beta}/B} \cdot \vec{n}_{\alpha\beta} \quad (1)$$

The weighting is computed in `micro.py` (Cromosim 2025b) as:

```

1  # Compute angular weighting factor
2  norm_Vdi = np.linalg.norm(Vd[i,:])
3  if norm_Vdi > 0:
4      theta_ij = np.arccos(np.dot(Vd[i,:], [eij_x,eij_y])/norm_Vdi)
5      omega_ij = lambda_ + (1-lambda_)*(1+np.cos(theta_ij))/2
6  else:

```


Where:

- $\lambda \in [0, 1]$ controls peripheral awareness (0 = strong frontal bias, 1 = equal perception in all directions)
- θ_{ij} is the angle between desired velocity $\vec{V}_d$ and interaction direction

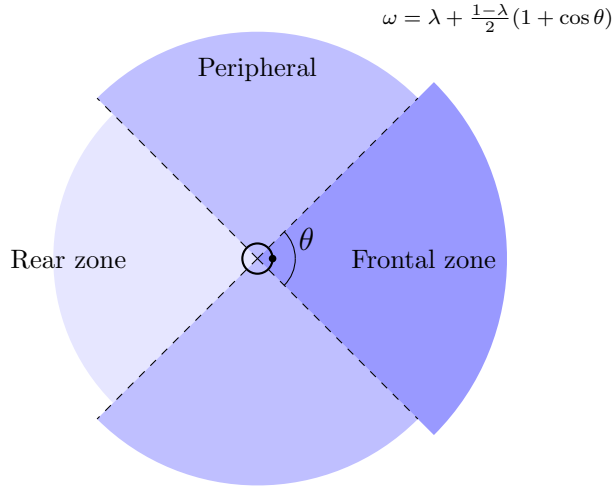


Figure 2: Extended perception field showing three zones of angular awareness. Awareness decreases from front to back based on the cosine-weighted factor ω_{ij} .

Physical Interpretation

The weighting function creates three distinct zones:

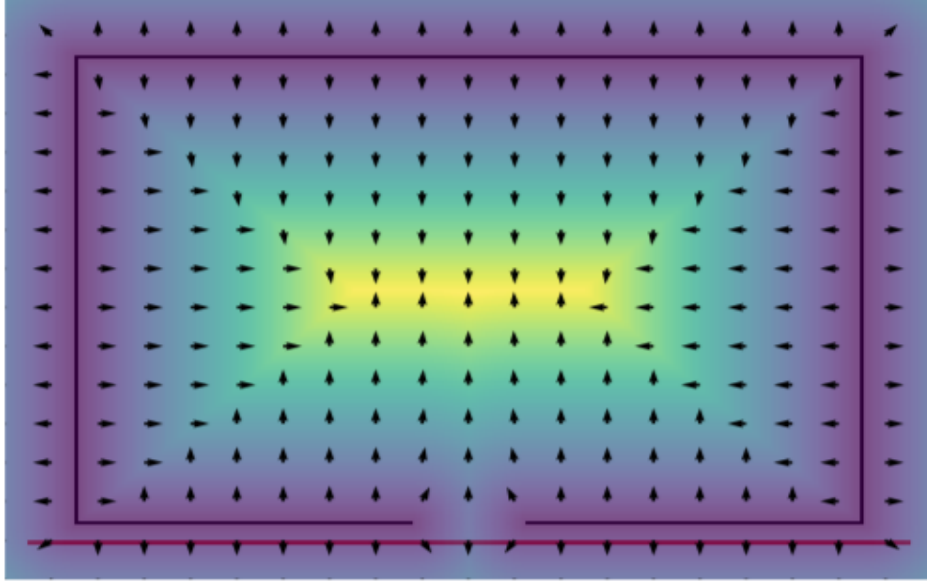
1. **Frontal zone** ($|\theta| < 45^\circ$): Full force perception
2. **Peripheral zone** ($45^\circ < |\theta| < 135^\circ$): Gradually decreasing awareness
3. **Rear zone** ($|\theta| > 135^\circ$): Minimal awareness (scaled by λ)

The default $\lambda = 0.5$ value matches experimental findings that humans maintain about 50% situational awareness behind them.

Handling Walls and Obstacles

Walls and obstacles use a similar repulsion system but with different strength settings. The forces work the same way, but walls don't move, so the calculations are simpler. This helps create natural-looking avoidance behavior where pedestrians smoothly steer around obstacles.

Figure 3: Distance from Wall



The colors in this graph represent how far away all points in the domain are from a wall. The arrows indicate the direction of the force that acts on a person at each point in the domain always prioritizing the nearest wall.

3.3 Managing Movement and Collisions

The theoretical speed limitation:

$$\frac{d\vec{r}_\alpha}{dt} = \vec{v}_\alpha(t) = \vec{w}_\alpha(t) \cdot g\left(\frac{v_\alpha^{\max}}{\|\vec{w}_\alpha\|}\right)$$

is implemented via quadratic programming:

```
1 # micro.py - Velocity projection
2 def projection(dt, xyrv, contacts, Vd, method="cvxopt"):
3     # Solves: minimize ||U - Vd||^2 subject to B·U >= (D - d_min)/dt
4     # Where B is the contact constraint matrix
5     if method == "cvxopt":
6         import cvxopt
7         solution = cvxopt.solvers.qp(Id, -V, cvxoptB, Z)
8         U = solution['x']
9     return U
```

The simulation makes sure pedestrians move realistically while avoiding impossible situations like walking through walls or other people. It does this by solving an optimization problem every time step that balances two goals:

1. Keeping movement as close as possible to what each pedestrian wants to do
2. Making sure no collisions happen

This calculation considers all the nearby interactions at once to find the best compromise. The system can use different solving methods depending on what works best for the situation. If one method has trouble, it automatically switches to a more reliable alternative.

The end result is smooth, natural-looking crowd movement where people reach their destinations while realistically avoiding bumps and jams.

3.4 Updating Pedestrian Positions

After computing the velocity field using optimization, Cromosim updates each pedestrian's position with the `move_people` function. This step integrates motion over time and handles interactions with virtual sensors.

```

1  # micro.py - Update people positions
2  def move_people(time, dt, people, sensors):
3      """
4      Updates the people positions according to the new velocities U. \
5      If there exists crosslines (i.e. sensors), computes also the id, time, \
6      direction and impact points for the individuals who cross the lines.
7      """
8      if (len(sensors) > 0):
9          xyrv_old = people["xyrv"].copy()
10         # Update people coordinates
11         people["xyrv"][:, 0] += dt*people["U"][:, 0]
12         people["xyrv"][:, 1] += dt*people["U"][:, 1]

```

The new position for each pedestrian is calculated using simple Euler integration:

$$\vec{r}_\alpha(t + \Delta t) = \vec{r}_\alpha(t) + \Delta t \cdot \vec{U}_\alpha(t)$$

where $\vec{U}_\alpha$ is the optimized velocity from the projection step.

This mechanism allows the simulation to track flow rates and detect bottlenecks or congestion by monitoring how many people pass through designated areas over time.

4 Methodology

The goal of this investigation is to understand the social force model and how it describes pedestrian dynamics. For this purpose we designed a series of experimental simulations to capture the effects that the different parameters of the SFM have on the behaviour of crowds, specifically in evacuation scenarios. The parameters in question are Agent Count (Number of individuals in the room), Door Width (size of the room exit), Desired Speed (Mean Desired Speed of the population), Repulsion Strength (Magnitude of Repulsive forces from other agents and walls), and Relaxation Time (characteristic time taken to return to desired velocity after deviation), all selected because they are the most defining and influential to the crowd behaviour in the SFM. Each parameter was assigned a base value that is kept constant when measuring changes in the other parameters. We then defined ranges of values to test for each parameter and iterated over them, keeping all else constant. Each parameter combination was simulated 10 times, measuring a series of evacuation metrics once per second of simulation. The mean and standard deviation of these metrics was calculated across the 10 repetitions to obtain reliable measurements of each metric for each parameter combination, thus defining the relationships between dependent and independent variables.

Cromosim had an example implementation of the social force model (Cromosim 2018) which we used as a base to build our simulation procedures and all the code in the remainder of the paper.

4.1 Domain Design

Using cromosim’s Domain class, the experimentation room geometry was designed as a simple rectangular area of dimensions 24x34 meters with an exit of variable width in the center of the bottom wall. The red Destination object is placed 1 meter below the exit, defining the points that the agents wish to reach. Starting from within the room, closest point of the destination is can only be reached by passing through the exit.

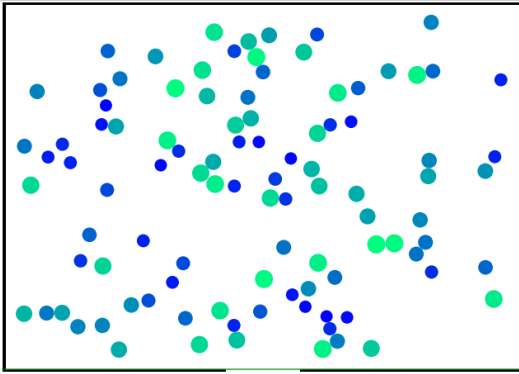


Figure 4: State after initialisation of simulation

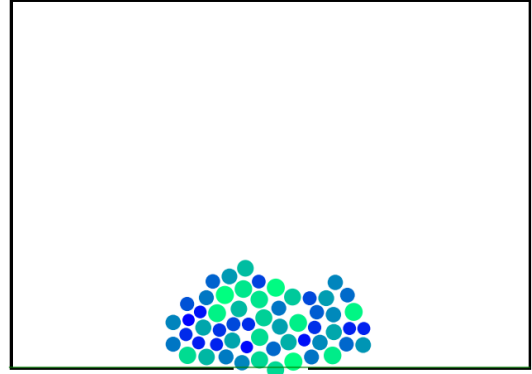


Figure 5: The evacuation process with accumulation at the door

4.2 Simulation Design

Domain Configuration

```
1 dom = Domain(name='room',
2     background='room_3_walls.png',
3     pixel_size=0.1)
4 dest = Destination(name='door',
5     colors=[[255,0,0]],
6     excluded_colors=[[0,0,0]])
7 dom.add_destination(dest)
```

The `domain.py` (Cromosim 2025a) library constructs the evacuation environment. The `Domain` class converts the PNG image into a navigable grid where white pixels are walkable areas and black pixels represent walls. The `Destination` class marks red pixels as exit targets and uses the Fast Marching Method to precompute distance fields for pathfinding. Walls are automatically detected from black pixels ([0,0,0] RGB values).

Agent Initialization

```
1 groups = [{
2     "nb": num_people,
3     "radius_distribution": ["uniform", 0.4, 0.6],
4     "velocity_distribution": ["normal", desired_speed, 0.1],
5     "destination": "door"
6 }]
7 people = people_initialization(dom, groups, dt)
```

The `micro.py` (Cromosim 2025b) library's `people_initialization` function creates pedestrian agents. It distributes agents randomly within the specified box area, assigning each a unique ID, radius (uniformly between 0.4-0.6m), and desired speed (normally distributed around the parameter value). The function resolves initial overlaps through iterative projection.

Social Force Computation

```
1 contacts = compute_contacts(dom, people["xyrv"], dmax=0.1)
2 Forces = compute_forces(F=repulsion_strength,
3     Fwall=repulsion_strength,
4     xyrv=people["xyrv"],
5     contacts=contacts,
6     lambda_=0.5, delta=0.08)
```

`compute_contacts` (from `micro.py`) (Cromosim 2025b) uses a cKDTree spatial query to detect all agent-agent and agent-wall pairs within 0.1m buffer distance. `compute_forces` then calculates repulsive forces using exponential decay with angular weighting ($\lambda=0.5$ reduces rear perception to 50%). The function returns force vectors for all agents.

Movement Integration

```

1 people["U"] = dt*(Vd-Uold)/relaxation_time + Uold + dt*Forces/mass
2 people = move_people(t, dt, people, sensors)

```

The velocity update combines three components: (1) relaxation toward desired velocity V_d with time constant τ , (2) persistence of current velocity U_{old} , and (3) force-driven acceleration. `move_people` (from `micro.py`) (Cromosim 2025b) advances positions using Euler integration ($x_{new} = x + U \cdot dt$) while recording sensor crossings through geometric line intersection tests.

4.3 Tested SFM Parameters

Formal definitions of the tested model parameters are as follows:

- **Agent Count:**

The number of people initialized in the room, whose initial positions are uniformly distributed across the space. In this case we used 100 people as our base simulation parameter, selected experimentally to guarantee crowding near the exit for most parameter combinations.

- **Door Width:**

This is the length of the exit of the room in meters. This parameter is key in how many people can exit at the same time. The value we used for this parameter in the simulation is 5, also selected experimentally.

- **Desired Speed:**

Each agent has a desired speed at which they want to walk. The speeds of agents are normally distributed with mean 1.2 m/s and standard deviation 0.1 m/s, for the base parameter. The standard deviation of the distribution is non-changing across simulations.

- **Repulsion Strength:**

The repulsion strength determines the magnitude of the repulsive forces enacted by other agents and the walls. This value is set at 2000 N, as per calibrated experimental research on the SFM (Kretz, Lohmiller, and Sukennik 2018)

- **Relaxation Time:**

The relaxation time specifies how fast an agents adapts their current velocity to their desired velocity. The lower the value the faster they react to the speed difference between the current and desired speed, and the more jittery the behaviour. The default value we used is 0.5 s.

4.4 Evacuation Metrics

To better capture the effects of the chosen model parameters on crowd dynamics we designed a set of evacuation metrics that can be used to understand the state of a crowd during evacuation. In total we decided to plot 5 model parameters against 4 evacuation metrics and try to understand each plots behaviour.

The following evacuation metrics are measured once per second of simulation – with exception of total evacuation time:

- **Evacuation Time:**

The time it takes from the start of the simulation until the last agent reaches the red destination line which is can only be reached by passing through the exit.

```
1  # Check if all people have evacuated
2  if people_count == 0:
3      if verbose:
4          print(f"All people evacuated at time {t}")
5          metrics["evacuation_time"] = t
6      break
```

- **Flow Rate:**

The flow rate is calculated as the number of people that exit the room every second of the simulation.

```
1  # Calculate flow rate (people per second) by counting how many people
   ↪ crossed the sensor in the last interval
2  evacuated_in_interval = len([et for et in exit_times if last_metric_time
   ↪ <= et < t])
3  flow_rate = evacuated_in_interval / metric_interval
4  metrics["flow_rate"].append((t, flow_rate))
```

- **Collision Rate:** The collision rate is measured in a similar way counting the number of contacts between agents, and also contacts with the walls, every second.

```
1  contacts = compute_contacts(dom, all_people[domain_name]["xyrv"], dmax)
2  metrics["collision_count"].append((t, len(contacts)))
```

- **Exit Density:** The number of people within a defined rectangular area (1m extension past door on both sides, and a total of 3m in height starting 0.5m below the exit) at the exit is counted every second.

```

1  # Calculate density near exit
2  people_at_exit = 0
3  for person in range(all_people[domain_name]["xyrv"].shape[0]):
4      x, y = all_people[domain_name]["xyrv"][person, 0:2]
5      if (exit_area["x_min"] <= x <= exit_area["x_max"] and
6          exit_area["y_min"] <= y <= exit_area["y_max"]):
7          people_at_exit += 1
8
9  density = people_at_exit / exit_area_size if exit_area_size > 0 else 0
10 metrics["exit_density"].append((t, density))

```

All evacuation metrics (except Evacuation Time) are then aggregated by taking the mean value of the simulation as an estimate of the metric for the simulation.

```

1 metrics["avg_flow_rate"] = np.mean([fr[1] for fr in metrics["flow_rate"]])
2 metrics["avg_collision_rate"] = np.mean([cc[1] for cc in
↪ metrics["collision_count"]])
3 metrics["avg_exit_density"] = np.mean([ed[1] for ed in
↪ metrics["exit_density"]])

```

As mentioned, to account for random variations in the experiment setup—such as initial positions, individual sizes, and desired speeds—each parameter combination was simulated 10 times with different random seeds. The results from these repetitions were then aggregated to calculate the mean and standard deviation of the evacuation metrics for each set of parameters:

```

1  # Calculate average metrics across repetitions
2  avg_evac_time = np.mean([res['evacuation_time'] for res in value_results])
3  std_evac_time = np.std([res['evacuation_time'] for res in value_results])
4
5  avg_flow_rate = np.mean([res['avg_flow_rate'] for res in value_results])
6  std_flow_rate = np.std([res['avg_flow_rate'] for res in value_results])
7
8  avg_collision_rate = np.mean([res['avg_collision_rate'] for res in
↪ value_results])
9  std_collision_rate = np.std([res['avg_collision_rate'] for res in
↪ value_results])
10
11 avg_exit_density = np.mean([res['avg_exit_density'] for res in
↪ value_results])
12 std_exit_density = np.std([res['avg_exit_density'] for res in value_results])

```

These aggregate values are presented graphically in the following results section.

5 Results

5.1 Variations of Number of People

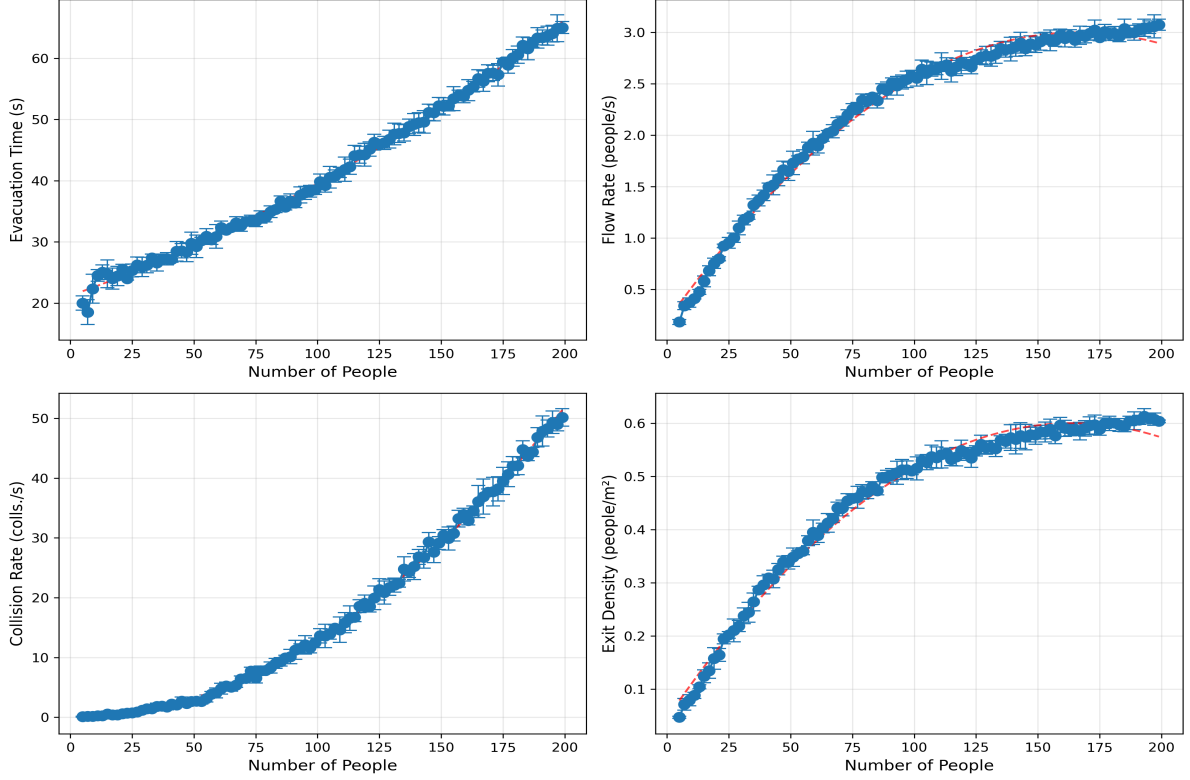


Figure 6: Evacuation Metrics vs. Agent Count

Evacuation time grows almost linearly with crowd size, rising from approximately 24.8s when 2 people are present to about 49.3s at 140 people. There is a slight convexity beyond 80 people, with each additional 20 people adding marginally more time—suggesting the onset of congestion around that threshold.

Average flow rate increases rapidly at first, climbing from roughly 0.78 people/s at 20 people to 2.34 people/s at 80 people, and then tapers off toward 2.85 people/s by 140 people. The concave-down shape indicates the door capacity is approached near 80-100 people, beyond which extra bodies yield diminishing throughput gains.

Collision rate remains low for small crowds—under 4 collisions/s up to 60 people—but accelerates sharply thereafter, reaching 8.3 collisions/s at 80 people, 12.9 collisions/s at 100 people, and ultimately 25.6 collisions/s at 140 people. This super-linear rise reveals a critical interaction regime emerging beyond roughly 60–80 people.

Exit density at the door increases from about 0.16 people/m² at 20 people to 0.48 people/m² at 80 people, then saturates toward 0.58 people/m² by 140 people. The concave-down trend reflects the physical limit of packing near the exit and suggests densities above 0.5 people/m² occur once the crowd exceeds roughly 100 individuals.

5.2 Variations of Door Width

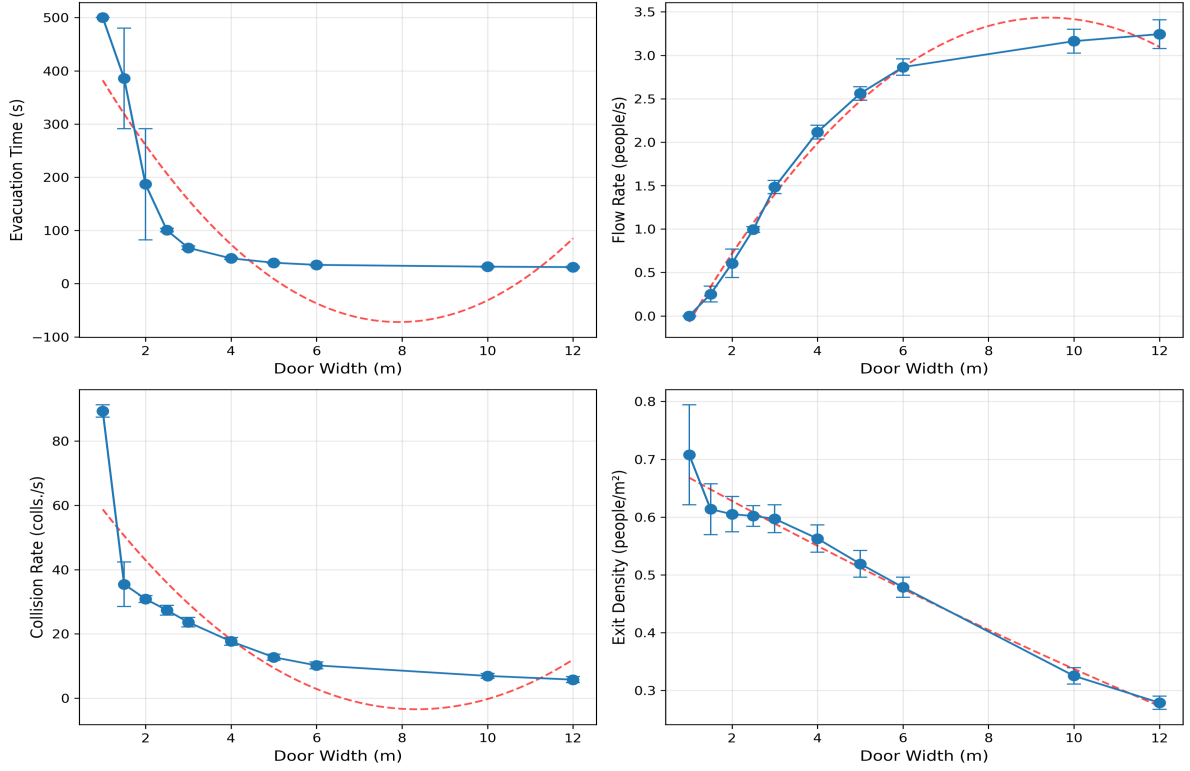


Figure 7: Evacuation Metrics vs. Door Width

Evacuation time drops steeply from around 500 s at a 1 m door to approximately 190 s at 2 m, then decreases more slowly to about 100 s at 3 m. The curve changes from convex to concave around 2–3 m, and beyond 4 m evacuation times plateau near 30–40 s, indicating little benefit from further widening.

Flow rate increases sharply with width, rising from zero at 1m to about 2.1 people/s by 4 m, then bending over around 4–5 m. By 6 m the flow approaches 2.8 people/s and plateaus near 3.2 people/s thereafter, marking the transition from door-limited to density-limited throughput.

Collision rate exhibits an exponential-like decay, plunging from roughly 90 collisions/s at 1 m down to 27 collisions/s at 2 m and 17 collisions/s at 4 m. The steepest decline occurs up to 3–4 m, after which the rate levels off around 10 collisions/s by 6 m and 6 collisions/s by 12 m, effectively eliminating aggressive contact past 4 m.

Exit density decreases almost linearly from about 0.70 people/m² at 1 m to 0.28 people/m² at 12 m, with no pronounced inflection. Each meter of additional width yields a roughly constant reduction in crowding, and densities fall below 0.5 people/m² only once the door reaches approximately 5 m.

5.3 Variations of Desired Speed

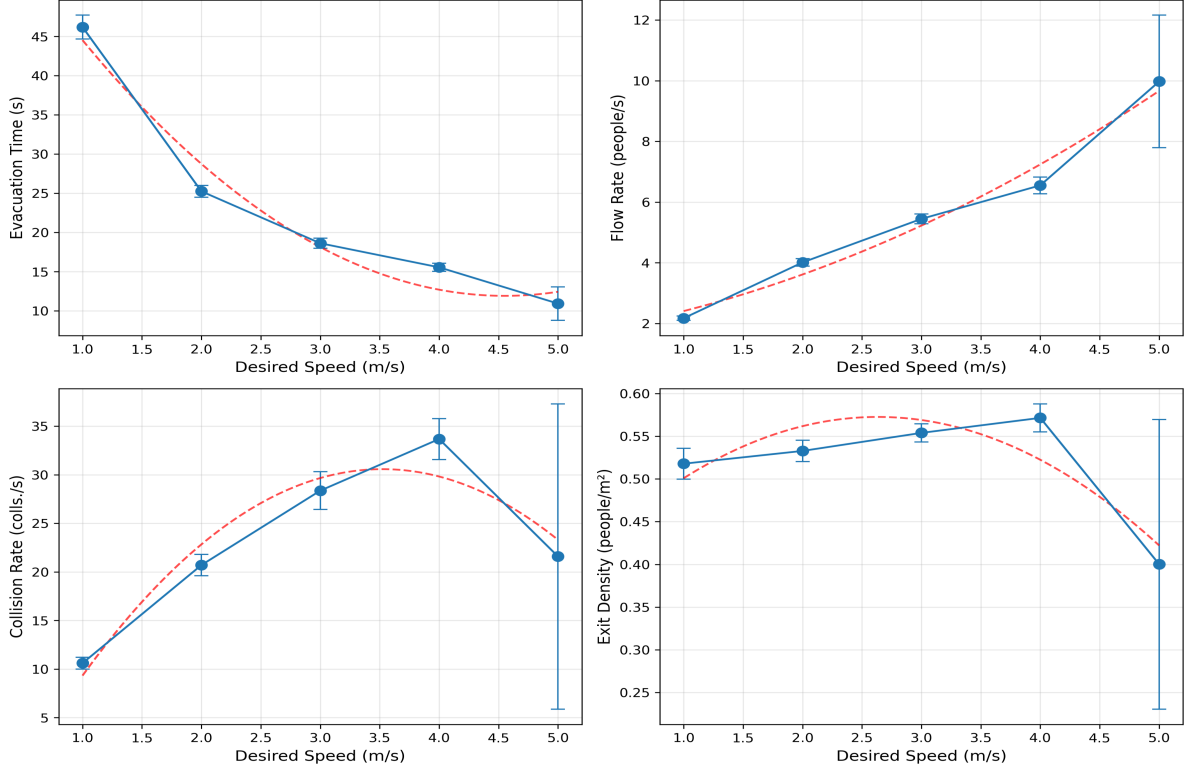


Figure 8: Evacuation Metrics vs. Desired Speed

Evacuation time falls dramatically from around 46 s at 1 m/s to 25.5 s at 2 m/s, then further to 15.7 s at 4 m/s, before dropping again to 10.9 s at 5 m/s. The curve is concave up between 2–4 m/s—each extra metre per second buys diminishing time savings—while the final sprint to 5 m/s produces a renewed, steep drop.

Flow rate scales almost linearly with desired speed up to 4 m/s, increasing from about 2.2 people/s at 1 m/s to 6.5 people/s at 4 m/s, and then surges to 10.0 people/s at 5 m/s. The absence of a plateau indicates that faster desired speeds continue to boost throughput, albeit with growing variance in the sprint regime.

Collision rate rises monotonically from roughly 11 collisions/s at 1 m/s to 33.5 collisions/s at 4 m/s, following a convex-up shape, and then unexpectedly drops to 21.6 collisions/s at 5 m/s. The peak around 4 m/s suggests maximal jostling occurs at brisk walking before the crowd fractures under sprint conditions.

Exit density at the door increases from about 0.52 people/m² at 1 m/s to 0.57 people/m² at 4 m/s, then falls sharply to 0.40 people/m² at 5 m/s. The curve is concave up until the density apex at 4 m/s and concave down thereafter, implying that sprint speeds disrupt the build-up of a dense queue.

The large standard deviations in these graphs at 5 m/s arise from simulation instability at large desired speeds. Past 4 m/s, with the remaining parameters left at their base values, we observed agent "explosions". People would rapidly fly out of the simulation environment upon collision, drastically reducing the number of individuals in the room

and skewing results to lower values for collision rate and exit density, while increasing flow rate.

5.4 Variations of Repulsion Strength

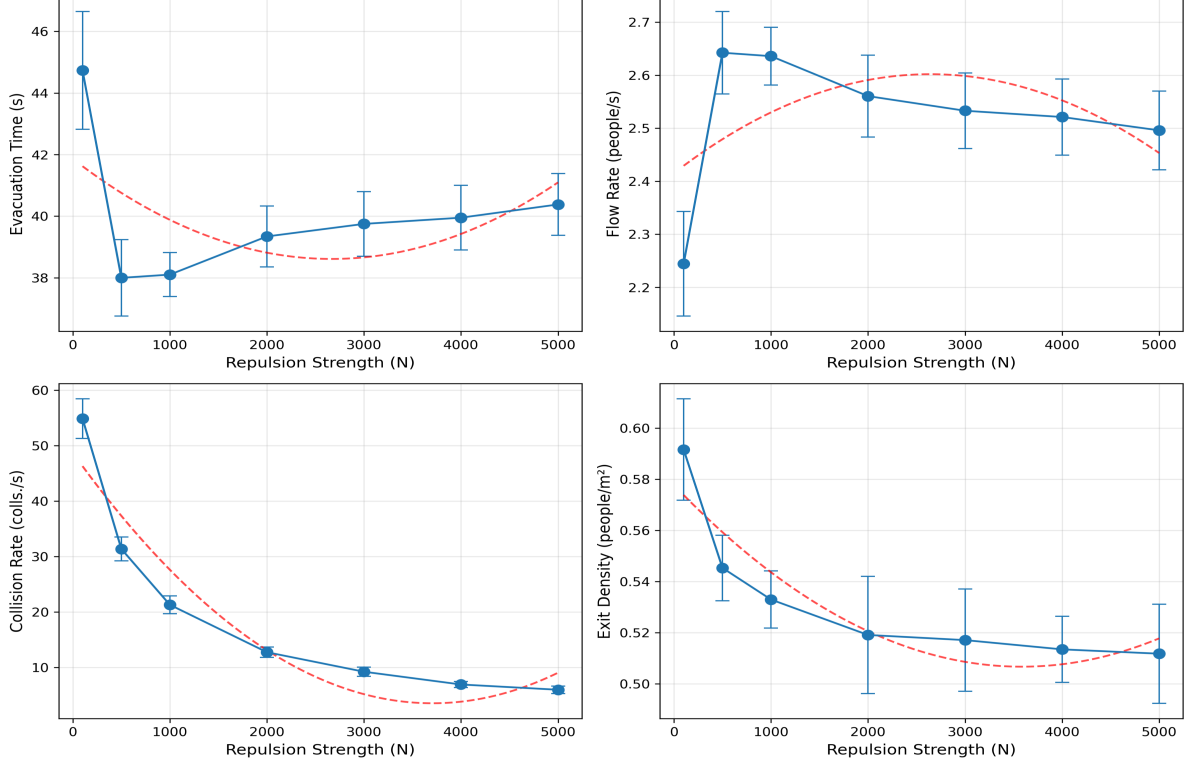


Figure 9: Evacuation Metrics vs. Repulsion Strength

Evacuation time decreases from approximately 44.7 s at 100 N to a minimum near 38.0 s at 500 N, then rises gradually to about 40.4 s by 5000 N. The U-shaped curve has its trough between 500–1000 N, indicating that moderate repulsion clears jams but excessive avoidance slows egress.

Flow rate climbs sharply from around 2.24 people/s at 100 N to a peak of 2.64 people/s at 500 N, holds steady at 2.63 people/s at 1000 N, and then tapers to 2.49 people/s at 5000 N. This inverted-U profile demonstrates optimal throughput at moderate repulsion and slight decline at very high forces.

Collision rate falls exponentially from about 55 collisions/s at 100 N to 21 collisions/s at 1000 N and 12.7 collisions/s at 2000 N, then plateaus near 6 collisions/s by 5000 N. The steepest decay occurs up to roughly 2000 N, beyond which additional repulsion yields negligible collision reduction.

Exit density decreases monotonically from roughly 0.59 people/m² at 100 N to 0.52 people/m² at 2000 N, and slowly to 0.51 people/m² at 5000 N. The gentle, concave-down decline indicates diminishing marginal gains in crowd thinning as repulsive forces grow large.

5.5 Variations of Relaxation Time

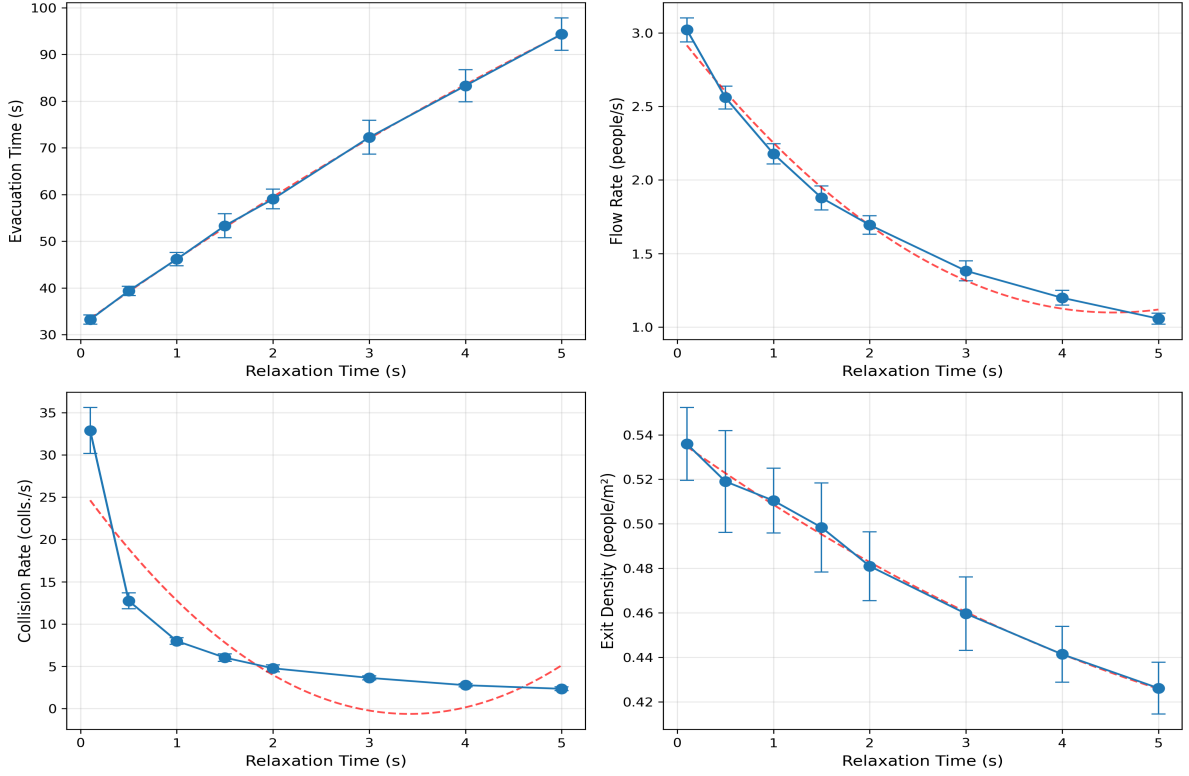


Figure 10: Evacuation Metrics vs. Relaxation Time

Evacuation time increases nearly linearly from about 33 s at $\tau = 0.1$ s to approximately 94 s at $\tau = 5$ s, with no evident inflection. This steady trend shows that slower agent reactions uniformly prolong total evacuation.

Flow rate declines sharply from roughly 3.0 people/s at $\tau = 0.1$ s to 2.2 people/s by $\tau = 1$ s, then more gradually to 1.05 people/s at $\tau = 5$ s. The concave-down shape highlights greatest sensitivity in the 0.1–1 s range and a plateauing tail beyond $\tau \approx 3$ s.

Collision rate drops from about 33 collisions/s at $\tau = 0.1$ s to 8 collisions/s at $\tau = 1$ s, then asymptotes near 2–3 collisions/s by $\tau = 5$ s. The steep decay up to 1 s followed by a flat tail indicates that rapid reactivity is crucial for avoiding contact, beyond which collisions are already minimal.

Exit density falls gently from approximately 0.54 people/m² at $\tau = 0.1$ s to 0.43 people/m² at $\tau = 5$ s in an almost linear fashion. The modest decline with no clear threshold suggests that slower reaction times only slightly reduce crowding at the door.

6 Discussion and Limitations

The present simulation study systematically varied five key parameters of the basic Social Force Model (SFM)—crowd size, door width, desired speed, repulsion strength, and relaxation time—to quantify their effects on four evacuation metrics: total evacuation

time, flow rate, collision rate, and exit-area density. Evacuation time grows almost linearly both when you add more people (because each new person adds a roughly constant delay) and when you slow down agents’ reactions (because everyone takes longer to reach full speed). Flow rate rises with crowd size and desired speed up to a capacity-limited plateau, while collision rate grows nonlinearly with both variables but declines with very wide doors. Exit-area density decreases monotonically with door width and repulsion strength and increases with crowd size and desired speed. Notably, moderate repulsion (500–1000 N) and moderate to high desired speeds (2–4 m/s) yielded optimal trade-offs between efficiency and safety (reducing collisions and maximizing flow-rate).

These trends align closely with both empirical observations and other simulation studies. The approximately linear growth of evacuation time and concave-down saturation of flow with crowd size mirror the fundamental diagram of pedestrian traffic, which shows speed and flow decelerating only after densities exceed about 1.5–2 people/m² (Bosina and Weidmann 2018). High collision rates in large crowds and narrow exits are consistent with experimental drills and VR-based evacuation tests, where physical contacts and turbulence surge once local densities surpass comfortable thresholds (Kretz, Lohmiller, and Sukennik 2018).

We also observed numerous discrepancies from existing works. The “faster-is-slower” effect—where raising desired speed beyond 3–4 m/s increases evacuation time by inducing clogging—has been widely reported in SFM work and animal experiments (Helbing and Molnár 1995), disagreeing with our experimental results which showed decreased times for increasing speeds. This is likely caused by the simulation instability, however, and is not regarded a fault of the social force model (Helbing and Molnár 1995). Empirical studies occasionally report sustained increases in flow for very wide exits beyond 6 m (Daamen and Hoogendoorn 2010), whereas our simulations showed plateauing near 3 p/s, which may be a consequence of unmodeled behavioral factors. Furthermore, real pedestrians exhibit anticipatory slowing and route choice that can mitigate the faster-is-slower effect at extreme urgencies (Oh and Park 2017), whereas the basic SFM enforces instantaneous adjustments that can exaggerate jamming.

Turning to the methodology, the SFM offers a compelling microscopic framework: its intuitive “forces” reproduce emergent phenomena such as lane formation, arching at bottlenecks, and density-dependent slowdowns (Helbing and Molnár 1995). Its continuous nature and ability to handle arbitrary geometries make it ideal for exploring “what-if” scenarios in complex layouts but omits cognitive decision-making, learning, and heterogeneity of individuals; it can generate non-physical oscillations and requires careful parameter calibration to avoid unrealistic overlaps or jittery motion (Kretz 2015). Cromosim implements the SFM in an accessible, open-source Python library, complete with visualization tools and sensor-based outputs (Cromosim 2018). While excellent for teaching and rapid prototyping, Cromosim’s 2D focus, limited documentation, and lack of built-in behavioral extensions constrain its use for rigorous evacuation planning.

Briefly, alternative approaches—cellular automata for fast, grid-based modeling and macroscopic continuum or compartment models for large-scale planning—offer complementary strengths but sacrifice individual detail (Richards 2016). Cognitive agent models that incorporate route choice, panic contagion, and group behaviors can address SFM’s representational gaps but at the cost of greater complexity and calibration demands.

7 Conclusions

This study demonstrates that the Social Force Model, when implemented in Cromosim, captures key qualitative trends of emergency evacuation dynamics: evacuation time and flow dependence on crowd size and speed, collision surges under high urgency or density, and the alleviating effects of wider exits and stronger personal-space repulsion. Our parameter sweeps reveal clear regime changes—such as optimal repulsion strength (500–1000 N) and desired speeds (2–4 m/s)—that minimize time and collisions while avoiding excessive queuing density.

Comparison with empirical and simulation-based literature shows broad agreement: the fundamental diagram’s flow limit, and jam-induced collisions emerge naturally. Observed discrepancies likely stem from real humans’ route choice, anticipation, and cultural heterogeneity, which the base SFM does not model.

Methodologically, while the SFM’s force-based abstraction excels at generating emergent crowd patterns, its neglect of cognition and decision-making mandates cautious interpretation of absolute metrics. Cromosim provides a convenient platform for exploring these dynamics, but its 2D scope and pedagogical orientation limit its use for detailed safety certification. For robust evacuation analysis, practitioners should calibrate SFM parameters against empirical trajectory data, consider cognitive-behavioral extensions, or employ complementary modeling approaches as appropriate.

Overall, the Social Force Model remains a foundational tool for understanding crowd phenomena, offering valuable insights into how key parameters influence evacuation performance—even as ongoing work seeks to integrate richer behavioral realism.

8 References

All the code used in this project can be found and downloaded from the following repository: https://github.com/marcnebelung/final_project_math_modeling

Bosina, Ernst and Ulrich Weidmann (2018). *Creating a generic model of the pedestrian fundamental diagram*. URL: https://www.strc.ch/2018/Bosina_Weidmann.pdf (visited on 05/18/2025).

Cromosim (2018). *Social Force Model*. Cromosim.fr. URL: https://www.cromosim.fr/example_micro.html#social-force-model (visited on 05/18/2025).

— (2025a). *cromosim/domain.py*. GitHub/cromosim. URL: <https://github.com/sylvain-faure/cromosim/blob/master/cromosim/domain.py> (visited on 05/18/2025).

— (2025b). *cromosim/micro.py*. GitHub/cromosim. URL: <https://github.com/sylvain-faure/cromosim/blob/master/cromosim/micro.py> (visited on 05/18/2025).

- Daamen, Winnie and Serge Hoogendoorn (Jan. 2010). “Capacity of doors during evacuation conditions”. In: *Procedia Engineering* 3, pp. 53–66. DOI: [10.1016/j.proeng.2010.07.007](https://doi.org/10.1016/j.proeng.2010.07.007). URL: <https://www.sciencedirect.com/science/article/pii/S1877705810004765> (visited on 05/18/2025).
- Helbing, Dirk and Péter Molnár (May 1995). “Social Force Model for Pedestrian Dynamics”. In: *Physical Review E* 51, pp. 4282–4286. DOI: [10.1103/physreve.51.4282](https://doi.org/10.1103/physreve.51.4282). URL: <https://arxiv.org/pdf/cond-mat/9805244> (visited on 05/18/2025).
- Kretz, Tobias (Nov. 2015). “On oscillations in the Social Force Model”. In: *Physica A: Statistical Mechanics and its Applications* 438, pp. 272–285. DOI: [10.1016/j.physa.2015.07.002](https://doi.org/10.1016/j.physa.2015.07.002). URL: <https://arxiv.org/pdf/1507.02566#:~:text=Under%20%E2%80%99Coscillations%E2%80%9D%20in%20this%20paper,the%20model%20parameter%20such%20that> (visited on 05/18/2025).
- Kretz, Tobias, Jochen Lohmiller, and Peter Sukennik (July 2018). “Some Indications on How to Calibrate the Social Force Model of Pedestrian Dynamics”. In: *Transportation Research Record: Journal of the Transportation Research Board* 2672, pp. 228–238. DOI: [10.1177/0361198118786641](https://doi.org/10.1177/0361198118786641). URL: https://www.researchgate.net/publication/322231209_Some_Indications_on_How_to_Calibrate_the_Social_Force_Model_of_Pedestrian_Dynamics (visited on 05/18/2025).
- Oh, Hyejin and Junyoung Park (Oct. 2017). “Main Factor Causing “faster-is-slower” Phenomenon during evacuation: Rodent Experiment and Simulation”. In: *Scientific Reports* 7. DOI: [10.1038/s41598-017-14007-6](https://doi.org/10.1038/s41598-017-14007-6). URL: <https://www.nature.com/articles/s41598-017-14007-6> (visited on 05/18/2025).
- Richards, Thomas (2016). *Pedestrian Simulation Software Review*. urban-analytics. URL: https://urban-analytics.github.io/dust/docs/ped_sim_review#cromosim-sl (visited on 05/18/2025).